

Mata Kuliah : WAD05 - Web Application Development
Judul Tugas : Technical Assignment 1: Web Security & JavaScript DOM Manipulation
Nama : Izhar Rahman Dwiputra
NIM : 25120300032

Soal 1 Perbandingan innerHTML dan textContent

innerHTML dan textContent sama-sama dapat digunakan untuk memasukkan isi ke dalam elemen HTML, tetapi keduanya memiliki cara kerja yang berbeda.

innerHTML memproses string yang diberikan sebagai kode HTML. Sebagai contoh, ketika elemen.innerHTML = "Halo", teks "Halo" akan ditampilkan dalam format tebal karena tag diproses oleh browser.

Sebaliknya, textContent memperlakukan seluruh nilai yang diberikan sebagai teks biasa. Jika digunakan dengan elemen.textContent = "Halo", browser akan menampilkan tulisan "Halo" apa adanya; tag tersebut tidak diproses sebagai HTML.

innerHTML sebaiknya digunakan ketika struktur HTML memang diperlukan dan sumber datanya telah dipastikan aman. Sementara itu, textContent lebih tepat digunakan untuk menampilkan teks, terutama teks yang berasal dari input pengguna, karena metode ini lebih aman terhadap serangan XSS seperti yang dibahas pada Soal 2.

```
elemen.innerHTML = "<i>" + komentarUser + "</i>";  
// berisiko apabila komentarUser mengandung skrip berbahaya
```

Perbandingan dengan versi yang lebih aman:

```
elemen.textContent = komentarUser;  
// aman karena nilai hanya ditampilkan sebagai teks dan tidak dieksekusi
```

Dengan demikian, apabila sumber data belum dapat dipastikan keamanannya, textContent merupakan pilihan yang lebih aman.

Soal 2 Skenario Cross Site Scripting XSS

Sebagai ilustrasi, sebuah blog memiliki kolom komentar. Ketika menampilkan komentar pengguna, pengembang tidak melakukan validasi terhadap input dan langsung menggunakan innerHTML untuk memasukkan komentar tersebut ke halaman.

Penyerang tidak selalu dapat menjalankan tag `<script>` secara langsung. Ketika tag tersebut disisipkan melalui innerHTML, browser tidak mengeksekusinya. Namun, penyerang dapat memanfaatkan elemen `<img>` dengan sumber gambar yang sengaja dibuat tidak valid, lalu menggunakan atribut `onerror` untuk menjalankan kode, seperti pada contoh berikut:

```

```

Ketika pengguna lain membuka halaman tersebut, elemen `<img>` gagal dimuat sehingga atribut `onerror` dijalankan secara otomatis di browser korban. Kode pada atribut tersebut dapat digunakan untuk mengirim cookie atau data sesi ke server milik penyerang.

Dampaknya cukup serius. Penyerang dapat mencuri sesi login (session hijacking) sehingga dapat mengakses akun korban tanpa mengetahui kata sandinya. Serangan ini juga dapat digunakan untuk mengarahkan korban ke situs phishing atau menyebarkan skrip ke komentar lain seperti worm.

Upaya pencegahan dapat dilakukan melalui beberapa langkah: (1) jangan mempercayai input pengguna secara langsung; lakukan validasi dan penyaringan terlebih dahulu; (2) gunakan `textContent`, bukan `innerHTML`, ketika menampilkan data dari pengguna, seperti yang dijelaskan pada Soal 1; dan (3) terapkan Content Security Policy (CSP) pada server sebagai lapisan perlindungan tambahan agar browser menolak skrip yang tidak tepercaya.

Prinsip dasarnya adalah jangan pernah mempercayai input pengguna secara langsung.

Soal 3 Live Search dengan createElement dan textContent

Kode berikut menerapkan fitur live search. Setiap kali pengguna mengetik pada kolom pencarian melalui event input, data disaring menggunakan `array.filter()`. Hasil penyaringan kemudian ditampilkan kembali menggunakan `createElement` dan `textContent`. Pendekatan ini tidak menggunakan `innerHTML` sehingga lebih aman terhadap XSS, sesuai dengan pembahasan pada Soal 2.

Berkas program: Izhar_25120300032_WAD05_Task2.html

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Cari Menu Warteg Dekat Kampus</title>
<style>
  body { font-family: 'Comic Sans MS', 'Segoe UI', sans-serif; max-width: 600px; margin:
40px auto; color: #222; }
  h1 { font-size: 22px; color: #d9480f; }
  #searchBox { width: 100%; padding: 10px; font-size: 16px; box-sizing: border-box;
margin-bottom: 12px; border: 2px solid #d9480f; border-radius: 6px; }
  #hasilList { list-style: none; padding: 0; }
  #hasilList li { padding: 10px; background: #fff3e0; margin-bottom: 6px; border-radius:
6px; }
  #infoJumlah { color: #888; font-size: 13px; }
</style>
</head>
<body>

<h1>Cari Menu Warteg Dekat Kampus</h1>
<p>Masih bingung menentukan menu makan? Ketik kata kunci pada kolom pencarian untuk
menyaring daftar menu tanpa memuat ulang halaman.</p>

<input type="text" id="searchBox" placeholder="Cari menu hari ini...">
<p id="infoJumlah"></p>
<ul id="hasilList"></ul>

<script>
  // daftar menu disimpan dalam array agar mudah dikelola
  const daftarMenu = [
    "Nasi + Ayam Goreng",
    "Nasi + Telur Balado",
    "Mie Goreng Spesial",
    "Nasi + Tempe Orek",
    "Nasi + Ikan Bawal Goreng",
    "Nasi + Sayur Sop",
    "Nasi + Rendang",
    "Nasi Uduk + Bihun",
    "Nasi + Perkedel + Tahu",
    "Nasi + Ayam Bakar"
  ];

  const inputSearch = document.querySelector("#searchBox");
  const listHasil = document.querySelector("#hasilList");
  const infoJumlah = document.querySelector("#infoJumlah");

  // menghapus daftar sebelum menampilkan hasil baru
  // removeChild digunakan karena innerHTML tidak boleh digunakan
  function bersihinList() {
    while (listHasil.firstChild) {
      listHasil.removeChild(listHasil.firstChild);
    }
  }

  // fungsi utama untuk menampilkan kembali daftar menu
  // createElement dan textContent digunakan untuk mengurangi risiko XSS (lihat Soal 2)
  function gambarUlangList(menuYangKetampil) {
```

```

    bersihinList();
    menuYangKetampil.forEach((menu) => {
        const itemBaru = document.createElement("li");
        itemBaru.textContent = menu;
        listHasil.appendChild(itemBaru);
    });
    infoJumlah.textContent = "Menampilkan " + menuYangKetampil.length + " dari " +
    daftarMenu.length + " menu";
}

// setiap input langsung diproses menggunakan array.filter tanpa menunggu pengguna
menekan Enter
inputSearch.addEventListener("input", (e) => {
    const kataKunci = e.target.value.trim().toLowerCase();
    const hasilFilter = daftarMenu.filter((menu) => {
        return menu.toLowerCase().includes(kataKunci);
    });
    gambarUlangList(hasilFilter);
});

// saat halaman pertama kali dibuka, tampilkan seluruh menu
gambarUlangList(daftarMenu);
</script>

</body>
</html>

```